HUMAN-READABLE, AI-EXECUTABLE

Agent Ledger Playbook

AI에게 일을 시킨 뒤, 나중에 다시 설명할 수 있게 만드는 저장 방법론. 사람이 읽으면 운영 철학이 보이고, AI에게 읽히면 실제 시스템이 세팅되도록 설계한 플레이북.

BACKBONE

Simon Willison의 공개 저장, 로그, 조회 워크플로를 실무 기준으로 번역

FRAME

Hashimoto의 하네스 관점으로 반복 실수 방지와 운영 구조를 연결

USE

Codex와 Claude Code가 같은 저장 규칙으로 일하도록 만드는 실행 팩

27

EDITORIAL PAGES DESIGNED FOR HUMAN READING AND AI INGESTION

08

EXECUTION PACK DOCUMENTS THAT CODEX OR CLAUDE CODE CAN DIRECTLY FOLLOW

05

MINIMUM FIELDS EVERY AI-ASSISTED TASK SHOULD LEAVE BEHIND

Free distribution, paid-level density
Version 1.0 · 2026-04-02

Good AI work starts with records, not memory.

READER NOTE

이 PDF는 사람이 읽는 설명서이면서, AI가 저장 시스템을 설계할 때 참고하는 상위 설계 문서입니다.

이 문서는 혼자 읽고 끝나는 전자책이 아니다. 사람이 읽으면 왜 기록이 필요한지 이해하게 만들고, AI가 읽으면 어떤 정보를 저장하고 어떤 규칙으로 운영해야 하는지 구조를 잡게 만드는 문서다.

중요한 원칙

실행 규칙의 정본은 함께 배포되는 Markdown Pack이다. 이 PDF는 철학과 운영 구조의 정본이며, 실제 구현, 템플릿, 체크리스트는 Markdown Pack이 담당한다.

WHO THIS IS FOR

AI를 매일 쓰지만 기록이 남지 않는 개인 작업자, 팀 리더, 개발자, 운영자

WHAT YOU GET

저장 항목 기준, 최소 로그 구조, Simon식 워크플로, Codex·Claude 적용법

WHAT THIS USES

Simon Willison의 공개 저장·로그·조회 워크플로를 중요한 실전 기준으로 사용

01

사람은 왜 기록이 필요한지 읽는다

02

AI는 MARKDOWN PACK을 읽고 저장 구조를 제안한다

03

팀은 주간 REVIEW로 규칙을 업데이트한다

이 PDF의 목표는 하나다. AI를 잘 쓰는 방법을 말하는 것이 아니라, AI 작업이 나중에도 설명 가능한 상태로 남게 만드는 것이다. 결국 문제는 모델의 지능이 아니라 기록의 부재에서 시작하는 경우가 훨씬 많기 때문이다.

HOW TO USE THIS PACKAGE

이 PDF로 개념과 기준을 이해한 뒤, 실제 적용은 `AGENTS.md`, `CLAUDE.md`, `pack/00~07`, `templates/*`를 프로젝트에 복사해 시작한다.

목차

1. 왜 AI 작업은 기록이 필요해지는가
2. 저장하지 않으면 생기는 7가지 문제
3. 최소한 무엇을 저장해야 하는가
4. 비개발자용 저장 방식
5. 개발자용 저장 방식
6. 가장 쉬운 시작법
7. Simon Willison은 어떻게 저장 시스템을 만들고 있나
8. Simon 방법론을 그대로 따라 할 때의 최소 구현
9. Mitchell Hashimoto 관점에서 보면 왜 이게 하네스 엔지니어링인가
10. 팀에 적용하는 순서
11. Codex에 적용하는 법
12. Claude Code에 적용하는 법
13. AI에게 이 패키지를 읽히는 프롬프트
14. 무엇을 저장하면 안 되는가
15. 로그를 남기되 위험하지 않게 만드는 법
16. 바로 쓰는 템플릿
17. 첫 7일 실행 플랜

이 플레이북의 가치는 정보량에 있지 않다. 무엇을 남기고, 무엇을 남기지 말아야 하며, 그 기록을 어떻게 다시 쓰게 만들지를 압축해서 담았다는 점에 있다.

다음 페이지부터는 Why - What - How - Execution - Guardrails - Assets 순서로 진행된다.

PART 1

Why: 왜 AI 작업은 기억이 아니라 기록으로 남아야 하는가

좋은 AI 사용은 좋은 답변에서 끝나지 않는다. 어떤 지시를 했고, 어떤 응답이 왔고, 어떤 도구가 돌았고, 누가 실행했는지가 남아 있어야 다음 품질이 올라간다.

왜 AI 작업은 기록이 필요해지는가

AI는 대화창에서 쓸 때 가장 편하다. 문제는 편한 만큼 흔적이 약하다는 점이다. 좋은 결과만 복사해 가면 작업의 이유와 과정은 사라진다.

좋은 답변은 남지만, 좋은 작업은 남지 않는다

대부분의 사용자는 AI를 "그때그때 답해주는 도구"로 쓴다. 그래서 최종 결과만 저장하고, 그 결과가 어떻게 나왔는지는 저장하지 않는다. 그런데 실제 업무에서는 최종 결과보다 과정이 더 중요할 때가 많다.

예를 들어, 글 초안이 이상하게 나왔을 때 원인을 찾으려면 최소한 아래를 다시 볼 수 있어야 한다.

- 내가 정확히 무엇을 요청했는가
- AI가 어떤 식으로 해석했는가
- 중간에 어떤 도구나 검색을 사용했는가
- 어떤 모델을 썼는가
- 누가 그 작업을 실행했는가

설명 가능한 작업이 결국 더 강하다

AI 사용은 점점 더 많은 조직에서 일상 업무가 되고 있다. 이때 중요한 것은 "한 번 잘 나온 결과"보다 "왜 이런 결과가 나왔는지 다시 설명할 수 있는 작업"이다. 설명 가능해야 수정이 가능하고, 수정 가능해야 팀의 품질이 올라간다.

> **핵심 문장**
> AI에게 일을 시켰다면, 결과만 남기지 말고 무슨 일이 일어났는지 나중에 다시 볼 수 있게 만들어라.

저장하지 않으면 생기는 7가지 문제

PROBLEM 01

같은 실수를 반복한다

이전 실패가 어떻게 만들어졌는지 모르기 때문에, 비슷한 작업에서 같은 오류가 다시 난다.

PROBLEM 02

좋은 프롬프트가 자산이 안 된다

잘 먹힌 요청을 다시 재사용하지 못한다. 결국 성과가 개인 기억에만 남는다.

PROBLEM 03

결과 품질을 비교할 수 없다

어떤 모델, 어떤 지시, 어떤 도구 조합이 좋았는지 비교할 수 없다.

PROBLEM 04

비용이 새는 지점을 모른다

무슨 작업에 어떤 모델을 얼마나 썼는지 모르면 최적화가 불가능하다.

PROBLEM 05

팀 학습이 누락된다

개인이 깨달은 좋은 사용법이 팀 규칙으로 번역되지 않는다.

PROBLEM 06

책임 소재가 흐려진다

누가 실행했는지, 누가 승인했는지 남지 않으면 운영이 취약해진다.

PROBLEM 07

보안 사고를 알기 늦어진다

반대로 아무 기준 없이 전부 저장하면 민감정보 사고가 날 수 있다. 그래서 저장 시스템은 기록성과 안전성을 함께 가져야 한다.

PART 2

What: 최소한 무엇을 저장해야 하는가

복잡한 시스템을 만들기 전에, 먼저 어떤 항목이 최소 필수인지부터 정해야 한다. 이 기준이 없으면 로그는 쌓여도 운영은 되지 않는다.

최소한 무엇을 저장해야 하는가

처음부터 다 저장할 필요는 없다. 하지만 너무 적게 저장하면 나중에 원인을 다시 볼 수 없다. 이 플레이북은 최소 필수 5가지를 제안한다.

REQUIRED 01

Prompt

내가 뭐라고 시켰는지. 나중에 결과가 이상할 때 가장 먼저 다시 봐야 하는 항목이다.

REQUIRED 02

Response

AI가 뭐라고 답했는지. 결과뿐 아니라 판단 흐름의 흔적이다.

REQUIRED 03

Tools

중간에 검색, 파일 읽기, 테스트 실행 같은 도구를 썼는지.

REQUIRED 04

Actor

누가 실행했는지. 개인 작업자라면 자신, 팀이라면 정확한 주체를 남긴다.

REQUIRED 05

Model

어떤 모델을 썼는지. 품질 비교와 비용 추적을 위해 필수다.

RECOMMENDED

Timestamp, Next Action, Artifact, Approval

시간, 다음 액션, 결과 파일, 승인 여부까지 남기면 운영 품질이 훨씬 좋아진다.

실무 기준

최소 저장 기준은 "뭘 시켰나, 뭐가 나왔나, 중간에 뭘 했나, 누가 실행했나, 어떤 모델이었나"의 다섯 질문에 답할 수 있어야 한다.

비개발자용 저장 방식

비개발자라고 해서 저장 시스템을 못 만드는 것은 아니다. 오히려 가장 좋은 시작은 복잡한 도구가 아니라 가볍게 계속 남기는 방식이다.

방법	좋은 점	한계	언제 적합한가
메모장	오늘 바로 시작 가능	검색과 구조화가 약함	개인 실험 초기
Markdown work log	가볍고 버전 관리 가능	분석 자동화는 약함	개인 작업자, 크리에이터
스프레드시트	필드별 비교와 정렬이 쉬움	긴 프롬프트/응답 관리가 번거로움	운영자, PM, 비개발 팀
노션 테이블	협업과 설명이 쉬움	정교한 구조화에는 한계	팀 위키와 같이 쓰는 경우

```
## 2026-04-02 15:20
- task: Simon 4월 글 Threads 초안
- actor: inabe
- model: gpt-5
- prompt: 사이먼 4월 글을 쉽게 설명해줘
- response: 운영, 추적, 권한 중심으로 정리
- tools: web_search
- next_action: 게시용 최종 다듬기
```

개발자용 저장 방식

개발자는 파일 기반 로그에서 시작해도 좋지만, 곧 구조화된 저장과 조회의 필요를 느끼게 된다. 이때 중요한 것은 멋진 시스템보다 단계적으로 올리는 것이다.

JSON은 자동화 가능한 최소 구조다

Markdown이 사람에게 좋다면 JSON은 도구에 좋다. CLI, 자동화 스크립트, 후속 분석, 대시보드 연결이 쉬워진다.

```
{
  "timestamp": "2026-04-02T15:00:00",
  "task": "summarize_simon_april_posts",
  "actor": "inabe",
  "model": "gpt-5",
  "prompt": "Simon 4월 글 핵심 정리",
  "response": "운영, 추적, 권한 중심",
  "tools": ["web_search"],
  "next_action": "write_threads_draft"
}
```

SQLite는 "다시 보기"를 가능하게 한다

JSON이 기록이라면 SQLite는 조회다. 작업별, 모델별, 사람별, 실패 패턴별로 다시 묻고 비교하려면 결국 queryable storage가 필요해진다.

바로 이 지점에서 Simon Willison의 방식이 강해진다. 그는 대화창을 저장하는 데서 멈추지 않고, 저장된 것을 다시 조회하고, plugin으로 usage와 actor까지 구조화한다.

PART 3

How: Simon의 공개 워크플로를 실무에 어떻게 옮길 것인가

이 파트의 목적은 영감을 주는 것이 아니다. 공개적으로 확인 가능한 Simon의 저장 구조를 복제 가능한 단계로 바꾸는 것이다.

가장 쉬운 시작법

대부분의 사람은 시작부터 너무 큰 시스템을 만들려고 한다. 그러면 아무것도 남기지 못한다. 이 플레이북은 가장 쉬운 5줄 로그부터 시작할 것을 권한다.

1. 작업 이름을 쓴다
2. 프롬프트를 적는다
3. 응답을 적는다
4. 도구와 모델을 적는다
5. 다음 액션을 적는다

> **좋은 시작의 기준**
> 가장 쉬운 시스템이 가장 좋은 시스템이다. 나중에 SQLite로 갈 수 있더라도, 오늘은 Markdown 파일 하나로 시작해도 충분하다.

Simon Willison은 어떻게 저장 시스템을 만들고 있나

이 장은 "사이먼이 대단하다"를 말하는 장이 아니다. 공개된 글과 릴리스에서 실제로 확인되는 저장 워크플로를 추려서, 우리가 복제 가능한 구조로 읽는 장이다.

1. LLM 실행을 대화창에서 끝내지 않는다

Simon의 `llm` 도구는 prompt와 response를 SQLite에 저장하는 방향을 택한다. 출발점부터 "채팅한 뒤 잊어버리는 사용법"이 아니라 "실행과 결과를 데이터로 남기는 사용법"이다.

2. 저장한 뒤 다시 조회한다

저장은 끝이 아니다. Simon은 저장된 SQLite를 Datasette로 다시 열어 보는 흐름을 제시한다. 이 구조는 중요하다. 로그가 있어도 다시 보기 어렵다면 학습으로 이어지지 않기 때문이다.

3. 2026년 4월에는 저장 범위를 더 넓혔다

4월 1일 공개 릴리스에서는 tool call loops, prompts/responses/tool calls logging, actor, allowed/default model, usage 기록 같은 항목이 반복해서 보인다. 즉, 단순 대화 저장에서 운영 로그 저장으로 계속 확장하고 있다.

> **Simon의 공개 워크플로를 한 줄로 줄이면**
>
> LLM 실행 → SQLite에 저장 → Datasette로 조회 → plugin으로 usage, tool calls, actor, model policy를 점점 더 구조화

Simon 방법론을 그대로 따라 할 때의 최소 구현

LEVEL 1

파일 로그 버전

작업마다 Markdown 블록을 쌓는다. 비개발자와 개인 작업자에게 가장 좋은 시작점이다.

LEVEL 2

JSON 버전

자동화와 구조화가 필요하면 JSON으로 옮긴다. 나중에 DB로 가기 쉬워진다.

LEVEL 3

SQLite 버전

모델별, 사람별, 작업별로 다시 묻고 비교해야 할 때 올린다.

LEVEL 4

조회 화면

저장만 하지 않고 다시 보는 화면을 붙인다. 그래야 운영과 리뷰가 가능해진다.

이 플레이북이 Simon을 적극 활용하는 이유는 여기에 있다. Simon은 저장을 "기록 습관"으로만 보지 않고, 저장과 조회를 하나의 운영 루프로 만든다. 이 점이 실무적으로 매우 강하다.

Mitchell Hashimoto 관점에서 보면 왜 이게 하네스 엔지니어링인가

Simon이 보여주는 것은 저장과 조회의 구조다. Mitchell Hashimoto의 언어로 바꾸면, 이것은 결국 에이전트가 같은 실수를 반복하지 않게 만드는 하네스의 일부다.

기록은 반복 실수를 줄이기 위한 전제다

기록이 없으면 같은 실수를 다시 보지 못하고, 다시 보지 못하면 규칙으로 바꾸지 못한다. Hashimoto가 강조하는 하네스는 결국 이런 반복 실수 방지 구조와 맞닿아 있다.

AGENTS.md와 로그는 서로 연결된다

AGENTS.md는 "앞으로 어떻게 일할 것인가"를 정의하고, 로그는 "실제로 어떻게 일했는가"를 남긴다. 이 둘이 분리되면 규칙은 이상적 선언으로 남고, 연결되면 실제 운영 기준이 된다.

팀에 적용하는 순서

단계	무엇을 한다	왜 필요한가
1. 개인 로그	최소 5개 필드로 기록 시작	기록 습관을 먼저 만든다
2. 공용 규칙	팀이 공통 필드를 정의	비교 가능한 데이터가 생긴다
3. 승인 지점	어디서 사람이 승인하는지 명시	운영 책임이 분명해진다
4. 민감정보 정책	무엇을 저장 금지할지 정함	보안 사고를 줄인다
5. 주간 리뷰	반복 오류와 유용한 패턴 추출	팀의 하네스가 진화한다

PART 4

Execution: Codex와 Claude Code에 실제로 적용하는 법

이 파트에서는 패키지를 AI에게 어떻게 읽히고, 실제 프로젝트에서 어떻게 세팅하게 만들지 다룬다.

Codex에 이 방법론을 적용하는 법

Codex에서는 AGENTS.md가 핵심이다. 패키지 루트의 AGENTS.md는 무엇을 저장하고 어떤 순서로 읽어야 하는지 상위 규칙을 정의한다.

1. 프로젝트 루트에 AGENTS.md를 둔다
2. AI가 먼저 읽을 pack 파일 순서를 정한다
3. work-log 템플릿이나 JSON 템플릿을 연결한다
4. 작업마다 최소 5개 필드를 남기도록 지시한다

```
이 폴더의 AGENTS.md와 pack 문서를 기준으로
내 프로젝트용 AI 작업 로그 시스템을 설계해줘.

조건:
- Simon Willison의 공개 워크플로를 핵심 기준으로 삼을 것
- 최소 저장 항목은 prompt, response, tools, actor, model
- 처음에는 work-log.md에 append하는 가장 쉬운 구조로 시작할 것
```

Claude Code에 이 방법론을 적용하는 법

Claude Code에서는 CLAUDE.md가 같은 역할을 한다. 중요한 것은 파일 이름이 아니라, AI가 읽는 우선순위와 저장 규칙이 일관되게 유지되는 것이다.

- CLAUDE.md에서 최소 저장 기준을 분명히 한다
- SIMON_METHOD_REFERENCE.md를 함께 읽게 한다
- pack/00~03을 기준으로 실제 로그 구조를 제안하게 한다
- 민감정보는 local-only와 redaction을 우선 제안하게 한다

> **핵심 원칙**
> Codex와 Claude Code가 다른 UI를 가질 수는 있어도, 저장 기준과 검토 기준은 같아야 한다.

AI에게 이 패키지를 읽히는 프롬프트

이 플레이북은 사람이 읽고 이해하는 동시에, AI에게 읽혀서 실제 저장 시스템을 만들게 하는 것을 목표로 한다. 아래 프롬프트는 그 목적에 맞춰 설계됐다.

BEST INPUT ORDER

가장 안정적인 방식은 `PLAYBOOK.pdf` 로 개념을 먼저 읽히고, 이어서 `pack/00_AI_INGESTION_NOTE.md`, `pack/01_SIMON_METHOD.md`, `pack/03_STORAGE_SCHEMA.md`, `pack/07_PROJECT_BOOTSTRAP_PROMPTS.md` 순서로 주는 것이다.

```
이 폴더의 PLAYBOOK과 Markdown Pack을 읽고
내 프로젝트용 AI 작업 로그 시스템을 설계해줘.

요구사항:
- Simon Willison의 공개 워크플로를 적극 반영
- 최소 필수 항목: prompt, response, tools, actor, model
- 민감정보는 저장하지 않거나 마스킹
- 처음에는 Markdown 또는 JSON으로 시작
- 필요하면 SQLite로 확장할 수 있는 경로 제안
- 사람도 이해할 수 있는 설명 포함
```

```
이 프로젝트는 비개발자와 개발자가 함께 사용한다.
가장 쉬운 저장 구조부터 제안하고,
7일 안에 운영이 시작될 수 있는 최소 실행 플랜을 만들어줘.
```

PART 5

Guardrails: 무엇을 저장하지 말아야 하는가

저장은 강력하지만, 무조건 다 저장하면 위험하다. 좋은 저장 시스템은 기록성과 안전성을 함께 갖는다.

무엇을 저장하면 안 되는가

DO NOT STORE

시크릿, 토큰, 비밀번호

가장 기본이다. 자동 저장 시스템에 들어가면 안 되는 1순위다.

DO NOT STORE

고객 원문 민감정보

필요하면 요약본 또는 마스킹된 형태만 남긴다.

DO NOT STORE

내부 비공개 자산 전체

필요한 필드만 추려 저장해야 한다.

DO NOT STORE

민감정보가 섞인 전체 프롬프트

가능하면 요약 또는 redacted prompt를 사용한다.

저장은 많을수록 좋은 것이 아니라, 다시 볼 가치가 있는 것만 안전하게 남기는 것이 중요하다.

로그를 남기되 위험하지 않게 만드는 법

- 기본은 local-first로 시작한다
- 민감정보는 저장 전 redaction을 먼저 고려한다
- 프로젝트별 저장 위치를 분리한다
- 내보내기 전에는 사람이 한 번 검토한다
- 모든 로그를 중앙 서버로 자동 전송하는 구조는 신중하게 검토한다

> **좋은 기본값**
> 처음에는 내 컴퓨터 안의 파일이나 SQLite로 시작하고, 나중에 정말 필요할 때만 공유 스토리지나 협업 도구로 확장하는 편이 안전하다.

PART 6

Assets: 바로 쓰는 템플릿과 7일 실행 플랜

이 마지막 파트는 바로 적용을 돕기 위한 실전 자산이다. 여기서 멈추지 말고, 실제 파일을 복사해 프로젝트에서 써야 한다.

바로 쓰는 템플릿

ASSET	용도	언제 먼저 쓸까
work-log.md	가장 쉬운 작업 기록	오늘 바로 시작할 때
ai-task.json	구조화된 작업 로그	자동화나 반복 작업이 늘 때
weekly-review.md	실패 패턴과 규칙 업데이트	일주일에 한 번 review 할 때
research-note.md	날짜와 팩트 중심 리서치 기록	딥리서치와 Threads 준비 시
team-policy.md	팀 공용 저장 규칙	여러 사람이 같은 AI 규칙을 써야 할 때

1. work-log.md

```
## {{timestamp}}
- task:
- actor:
- model:
- prompt:
- response:
- tools:
- next_action:
- artifact:
```

2. ai-task.json

```
{
  "timestamp": "",
  "task": "",
  "actor": "",
  "model": "",
  "prompt": "",
  "response": "",
  "tools": [],
  "next_action": "",
  "artifact": ""
}
```

3. weekly-review.md

첫 7일 실행 플랜

DAY	실행	목표
Day 1	work-log 시작	기록 습관 만들기
Day 2	model 기록	품질과 비용 비교 기반 만들기
Day 3	tool 기록	중간 행동 로그 확보
Day 4	actor 기록	실행 주체 분명히 하기
Day 5	next_action 기록	로그가 다음 작업으로 이어지게 만들기
Day 6	weekly review 초안	반복 오류 찾기
Day 7	AI에게 시스템 설계 맡기기	Markdown -> JSON/SQLite 확장 경로 만들기

7일 뒤의 목표

완벽한 시스템이 아니라, 최소한 "무슨 일을 시켰고, 어떤 결과가 나왔고, 누가 무엇을 했는지" 다시 볼 수 있는 상태를 만드는 것이다.

APPENDIX A

사람용 요약

이 플레이북을 한 문장으로 요약하면 이렇다. AI를 잘 쓰는 사람은 기억이 좋은 사람이 아니라, 기록을 남기는 사람이다.

- 최소 저장 항목은 prompt, response, tools, actor, model이다
- Simon Willison의 공개 워크플로는 저장과 조회를 함께 보게 만든다
- Hashimoto의 하네스 관점은 반복 실수 방지 구조로 이어진다
- 처음에는 Markdown 파일 하나로 시작해도 충분하다
- 민감정보는 기본적으로 저장하지 않거나 마스킹해야 한다

APPENDIX B

AI Ingestion Note

이 PDF는 상위 설계 문서다. 실제 규칙과 템플릿은 Markdown Pack을 기준으로 한다. AI는 이 문서를 읽은 뒤 Simon Method, Capture Rules, Storage Schema, Workflow Setup, Review And Feedback, Privacy Redaction, Bootstrap Prompts를 우선 읽고 시스템을 제안해야 한다.

APPENDIX C

비개발자와 개발자의 시작 경로

사용자	먼저 열 파일	추천 템플릿
비개발자	`PLAYBOOK.pdf` → `templates/work-log.md` → `examples/nondev-ops-example.md`	`work-log.md`, `weekly-review.md`
개발자	`PLAYBOOK.pdf` → `AGENTS.md` 또는 `CLAUDE.md` → `pack/03_STORAGE_SCHEMA.md`	`ai-task.json`, `weekly-review.md`

APPENDIX D

Sources And License

이 패키지의 Simon 관련 설명은 아래 1차 출처를 기준으로 작성했다.

- `llm` repository: https://github.com/simonw/llm

- `Language models on the command-line` handout: https://github.com/simonw/language-models-on-the-command-line
- Simon Willison April 2026 archive: https://simonwillison.net/2026/Apr/

이 패키지는 `MIT License`로 무료 공개된다. 사용, 복사, 수정, 배포가 가능하며, 재배포 시 라이선스 고지를 함께 포함하면 된다.

APPENDIX E

Changelog

v1.0 (2026-04-02): Initial public playbook body with Simon-first architecture and Codex/Claude execution guidance.